GRADUATION PROJECT DOCUMENTATION

SupplyMind AI

AI-Powered Demand Forecasting & Inventory Optimization Platform

Table of Contents

Project Information

Role	Name	Primary Responsibility
Team Leader · Full Stack AI Engineer	Ibrahim Abdelsttar Abdelgawad	FastAPI backend, PostgreSQL, Docker, authentication, deployment, LLM Documentation
LLM Engineer	Kenzy Walid Sorour Hosny	AI insights, LLM reasoning, report generation
Data Analyst	Rahma Shaaban Elhusseiny Shaaban	Data pipeline, feature store, dashboard pages
ML Engineer	Karim Ayman Abdelgaber Deif	XGBoost training & evaluation, model artifacts
MLOps Engineer	Ali El Shaarawy	Drift detection, retraining pipeline, model monitoring
RAG Engineer	Ali Ehab Mossad Abdelghany	Inventory page, RAG system, alert engine

1. Executive Summary

SupplyMind AI is an AI-powered supply chain intelligence platform that unifies demand forecasting, inventory optimization, explainable AI insights, real-time alerting, reporting, and MLOps monitoring into a single web application. What began as a university capstone project has been engineered toward a commercially credible product: a full-stack system with a typed React frontend, a modular FastAPI backend, a trained gradient-boosting forecasting model, a retrieval-augmented generation (RAG) knowledge layer, and a production-oriented deployment stack.

The platform is built around a simple thesis: supply chain teams do not just need a forecast — they need a forecast they can trust, inventory recommendations they can act on, and an explanation they can bring to a stakeholder meeting. Every major feature in the system is designed to close the gap between raw model output and a decision a human is willing to make.

1.1 Core Capabilities

- Multi-horizon demand forecasting (1 / 3 / 6 months) powered by an XGBoost model, reporting $R^2 = 0.9984$ and WAPE = 1.51% on the project's evaluation data.
- Inventory optimization: automated Economic Order Quantity (EOQ), safety stock, and reorder point calculations, lead-time aware.
- Explainable AI (XAI): SHAP-based feature attribution translated into plain-language business narratives by an LLM layer.
- Retrieval-augmented generation: a ChromaDB vector store plus sentence-transformer embeddings ground the AI copilot's answers in the platform's own operational data.
- Guardrailed LLM access: input/output sanitization, prompt-injection defenses, rate limiting, and tenant isolation wrap every AI-facing endpoint.
- MLOps: MLflow model registry, drift detection, and retraining triggers, with optional LangSmith tracing for agent observability.
- Bilingual, accessible UI: full English/Arabic interface with right-to-left (RTL) layout support and a neumorphic design system.

1.2 Project Status

The system is in active development with the core product experience implemented end to end: authentication, the ML pipeline, the API surface, the AI orchestration/RAG layer, and the frontend dashboards all function against real (synthetic) business data. The team has also produced a formal business plan and market-sizing workbook as part of positioning the platform for commercial viability beyond the academic submission. Outstanding gaps — billing, multi-tenant data isolation, a broader dataset, and IP/licensing cleanup — are catalogued in Section 10.

1.3 Headline Metrics

The README surfaces the following headline figures on the product dashboard. The forecast accuracy figure is a measured result from the project's own model evaluation; the remaining operational-impact figures function as illustrative dashboard KPIs over the demo dataset rather than independently benchmarked outcomes, and should be treated accordingly in any external-facing pitch.

Metric	Figure	Status
Forecast accuracy	99.84% (R^2 = 0.9984, WAPE = 1.51%)	Measured — this project's evaluation set
Stock-out risk	−31%	Illustrative dashboard KPI over demo data
Inventory cost	−22%	Illustrative dashboard KPI over demo data
On-time delivery	98.7%	Illustrative dashboard KPI over demo data
Planning time	−60%	Illustrative dashboard KPI over demo data
Working capital	+18%	Illustrative dashboard KPI over demo data

2. Problem Statement & Solution

2.1 The Challenge

Manufacturers and distributors that run supply chain operations on spreadsheets and manual judgment face a recurring set of failures:

- Inaccurate demand forecasting leads to inefficient production and purchasing plans.
- Excess inventory ties up working capital that could be deployed elsewhere in the business.
- Frequent stockouts damage customer satisfaction and forfeit revenue.
- Black-box AI outputs cannot be justified to operations managers or executives who need to act on them.
- Manual, reactive inventory planning does not scale as product catalogs and store counts grow.
- There is no real-time visibility into operational risk until it becomes a problem.

2.2 The SupplyMind AI Response

Challenge	Platform Response
Inaccurate forecasts	XGBoost ensemble forecasting, $R^2 = 0.9984$
Overstock	Automated EOQ and safety-stock optimizer
Stockouts	Predictive stockout/overstock alert engine
Black-box AI	SHAP explainability layer + LLM narrative generation
Manual planning	Automated retraining pipeline and drift detection
No visibility	Real-time MLOps monitoring dashboard

> *The forecast-accuracy and business-impact figures throughout this document originate from the project's own model evaluation and the README's reported metrics. Where a claim is a planning assumption rather than a benchmarked fact (e.g. cost-savings percentages used in the business plan), it is labelled as such — see Section 9.*

3. System Architecture

SupplyMind AI follows a layered architecture: a React single-page application communicates with a FastAPI backend over a REST API; the backend orchestrates a machine-learning pipeline, a RAG knowledge layer, and an LLM gateway; and the whole system is containerized for both development and production deployment.

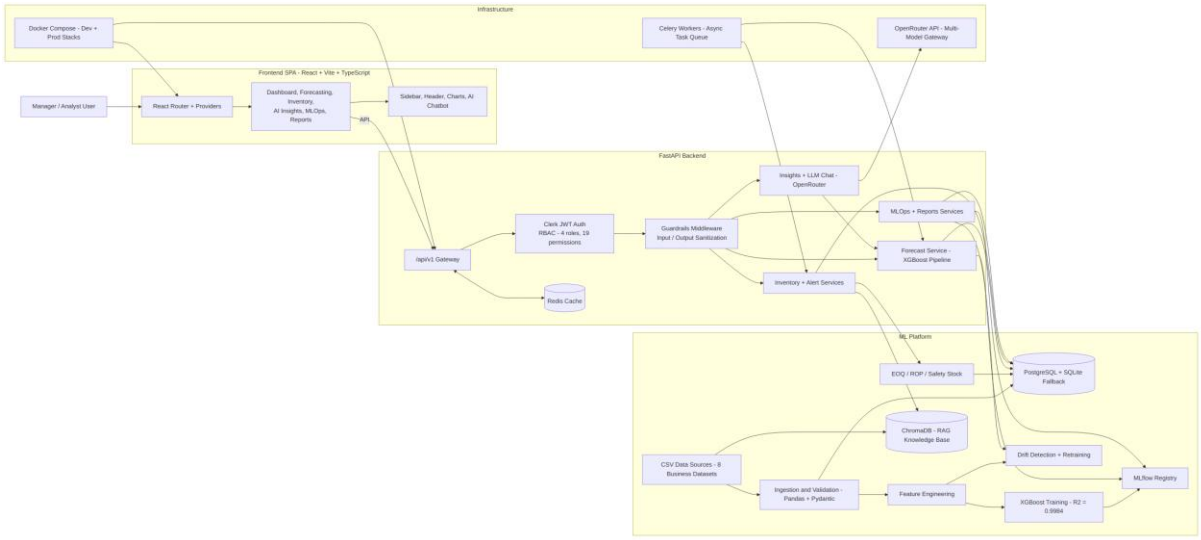


Figure 3.1 — Full system architecture: frontend, backend, ML platform, and infrastructure layers.

3.1 Architectural Layers

Frontend SPA

A React 18 + TypeScript single-page application built with Vite. Pages cover the Dashboard, Forecasting, Inventory, AI Insights, MLOps, and Reports workflows, backed by shared UI (sidebar, header, chart components, and an AI chatbot widget). State is split between server state (React Query) and app-level context providers (theme, currency, date range, auth).

FastAPI Backend

A modular FastAPI application exposes the /api/v1 surface behind Clerk JWT authentication and a role-based access control (RBAC) layer (4 roles, 19 permissions). A guardrails middleware chain sits between authentication and every business-logic service, and a Redis cache sits alongside the gateway for response caching.

ML Platform

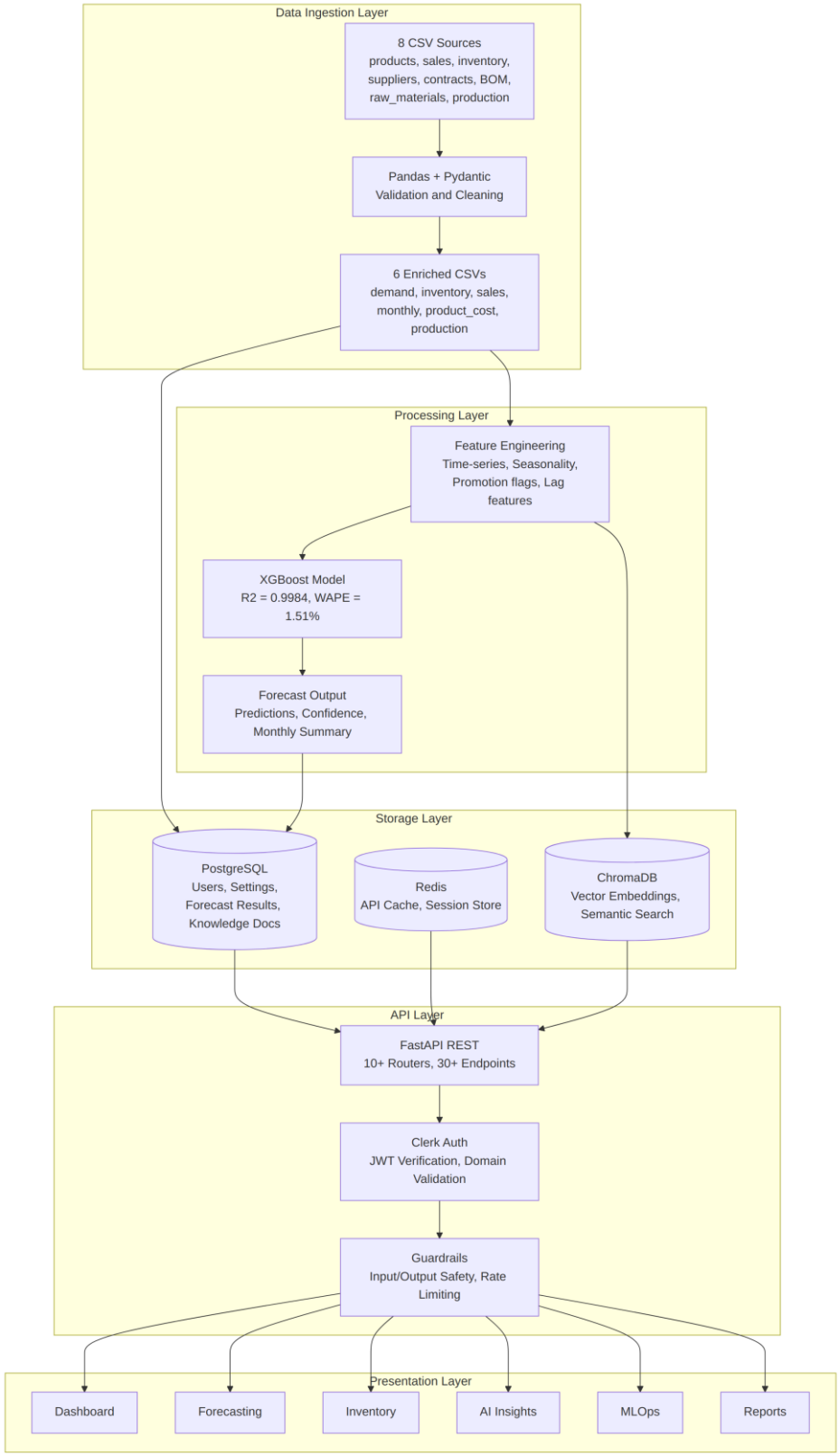
Eight raw CSV business datasets are ingested and validated with Pandas and Pydantic, transformed through a feature-engineering stage, and used to train the XGBoost forecasting model. Trained artifacts are versioned through an MLflow registry, and a drift-detection component watches for retraining triggers. Inventory optimization (EOQ / reorder point / safety stock) runs against the same data layer, and a ChromaDB vector store provides the RAG knowledge base.

Infrastructure

OpenRouter provides a multi-model LLM gateway; Celery workers handle asynchronous ML, RAG, and report-generation tasks; and Docker Compose defines both a development stack and an extended production stack (adding Redis, Celery worker, and Celery beat).

3.2 End-to-End Data Flow

The diagram below traces a single data flow from raw CSV ingestion through to the six presentation surfaces: Dashboard, Forecasting, Inventory, AI Insights, MLOps, and Reports.



4. AI & Machine Learning Pipeline

The forecasting pipeline runs as a closed loop: raw data becomes features, features train a model, the model is evaluated and registered, and drift signals feed back into retraining.

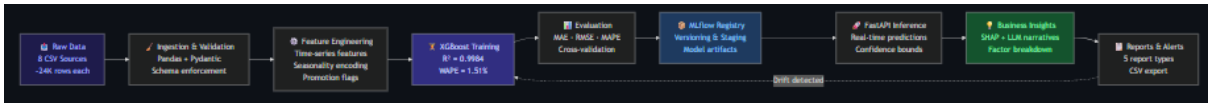


Figure 4.1 — Demand forecasting pipeline, from raw data to reports and alerts, with drift-triggered retraining.

4.1 Model Performance

Metric	Value	Description
R^2	0.9984	Coefficient of determination on the evaluation set
WAPE	1.51%	Weighted Absolute Percentage Error
Forecast horizons	1 / 3 / 6 months	Multi-horizon, product- and store-level predictions
Confidence intervals	Yes	Probabilistic bounds returned alongside point forecasts

4.2 Explainability (XAI)

Every forecast can be decomposed with SHAP (SHapley Additive exPlanations) into per-feature contributions — seasonality, promotions, historical demand, and other engineered features. These SHAP values are not surfaced as raw numbers; they are passed to the LLM layer, which converts them into a plain-language narrative ("demand is up largely due to seasonal effects and an active promotion") suitable for a non-technical stakeholder.

4.3 AI Orchestration Layer

Rather than routing every conversational request to a single generic LLM call, the platform runs an intent-based multi-agent orchestration layer in front of the LLM. Incoming chat and copilot requests pass through input guardrails, then an intent classifier that scores the query against eight possible intents (inventory, forecast, customer_support, documentation, reports, executive_insights, settings, unknown). If the classifier's confidence falls below 0.70, the system asks the user to clarify rather than guessing which agent should respond.

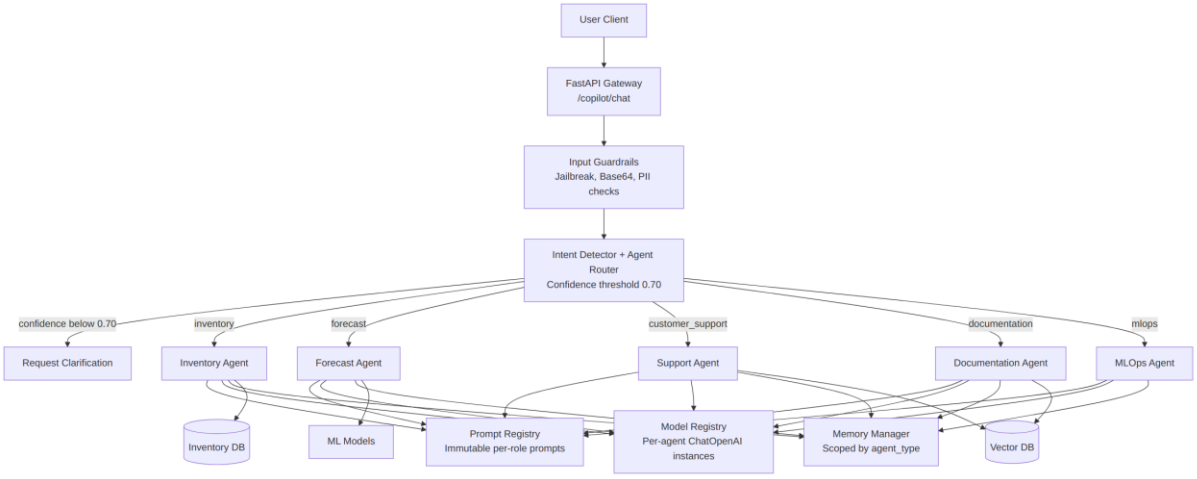


Figure 4.2 — AI orchestration layer: intent routing to isolated, specialized agents.

Above the confidence threshold, the query is handed to one of five specialized agents, each running its own restricted tool set:

Agent	Responsibility
Inventory Agent	Business-rule evaluation, knowledge-builder snapshots, and top-K semantic search over inventory context.
Forecast Agent	Explains seasonal demand, lead-time predictions, and model trends via generate_forecast and search_forecast_knowledge tools.
Support Agent	Answers dashboard/UI/settings questions from documentation FAQ indexes; explicitly restricted from data-modifying actions or reading inventory data.
Documentation Agent	Answers pricing, feature, and onboarding questions from product guides.
MLOps Agent	Tracks system memory, pipeline drift, and model accuracy.

Each agent is logically isolated even though all agents share a single OpenRouter API key for billing consolidation: the Model Registry instantiates a separate client per agent role with independent temperature, token, timeout, and retry settings; the Prompt Registry supplies immutable per-role system prompts to prevent context pollution; and memory and knowledge-document lookups are scoped strictly by agent_type and source_type, so one agent cannot read another's conversational memory or documents.

4.4 Retrieval-Augmented Generation (RAG)

The RAG layer grounds the AI copilot's answers in the platform's own operational documents and data rather than relying purely on the LLM's parametric knowledge. Text is embedded using the sentence-transformers all-MiniLM-L6-v2 model and stored in ChromaDB; queries are answered via hybrid retrieval — vector similarity combined with keyword (BM25) search — before being passed to the LLM as grounding context.

4.5 MLOps

- MLflow model registry: versioning and staging of trained model artifacts.

- Drift detection: automated monitoring of feature and prediction drift, feeding retraining triggers.
- Full LangSmith tracing across all orchestrator agents and service-level LLM call sites, for latency, token-count, and routing-accuracy observability.
- System resource gauges: operational health surfaced directly in the MLOps dashboard page.

4.6 LLM Optimization Strategy

Running LLM-backed features at scale is a cost and latency problem as much as a modeling problem. The platform implements eight concrete optimizations:

Strategy	Impact
AI Orchestration Layer	Strict per-agent data isolation with robust intent-based routing
Duplicate context removal	~50% fewer input tokens per call
Intelligence Engine consolidation	Eliminated duplicate database queries across services
Token budget enforcement	Per-feature input/output token budgets prevent runaway context
Agent iteration limits	Tool-call loops capped at 3 rounds
Response caching	1-hour TTL, SHA-256-keyed cache on repeated RAG queries
LLM observability	Full call tracking (stats, recent calls, cache hit/miss)
Streaming (SSE)	Real-time token streaming; first-token latency ~2 seconds

5. Authentication, Authorization & Guardrails

5.1 Authentication Flow

Authentication is handled by Clerk. Users sign in through a custom neumorphic Clerk UI; Clerk issues a JWT, which the backend verifies against Clerk's public JWKS. A domain check and a role check gate access before any protected route is reached.

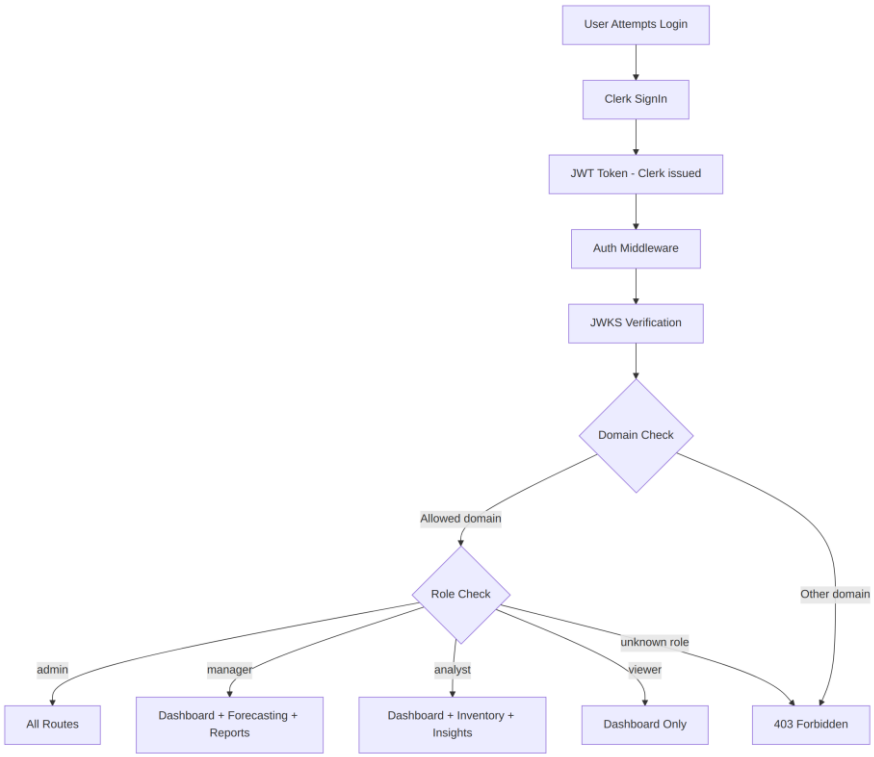


Figure 5.1 — Authentication and role-based access control flow.

5.2 Role-Based Access Control

Role hierarchy: admin > manager > analyst > viewer. Permissions are enforced per route across 4 roles and 19 distinct permissions. Manager and Analyst have converged to near-identical read/generate access, differing mainly in inventory write privileges; Viewer has been hardened to strictly read-only (no chatbot, write triggers, or generation).

Role	Dashboard	Inventory	AI Insights	Forecasting	Reports	MLOps	Settings
Admin	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Manager	Yes	Manage & Apply	View & Generate	View & Generate	View, Generate & Download	No	Yes
Analyst	Yes	View Only	View & Generate	View & Generate	View, Generate & Download	No	Yes

Role	Dashboard	Inventory	AI Insights	Forecasting	Reports	MLOps	Settings
Viewer	Yes	View Only	View Only	No	No	No	Yes

5.3 Guardrails System

Every AI-facing request passes through a layered guardrails pipeline before reaching business logic, and every response passes back through it before reaching the user.

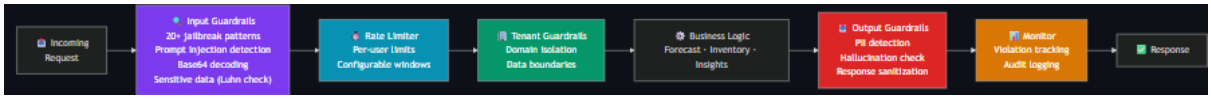


Figure 5.2 — Guardrails pipeline: input validation, rate limiting, tenant isolation, and output sanitization.

- Input guardrails: 20+ jailbreak/prompt-injection patterns, forbidden-topic filtering, adversarial-input detection, Base64 decoding checks.
- Output guardrails: PII detection and response sanitization before content reaches the client.
- RAG guardrails: RAG-specific input/output safety checks.
- Forecast guardrails: validation of forecast requests and confidence bounds.
- Agent guardrails: behavioral constraints on LangGraph agent actions.
- Tenant guardrails: domain isolation and data-boundary enforcement.
- Rate limiter: configurable per-user request limits.
- Monitor: violation tracking and audit logging across all of the above.

6. Data Model

The relational schema captures the full business domain — products, sales, inventory, production, suppliers, contracts, raw materials, and bills of materials — alongside the platform's own operational entities: users, forecasts, model runs, alerts, and reports.

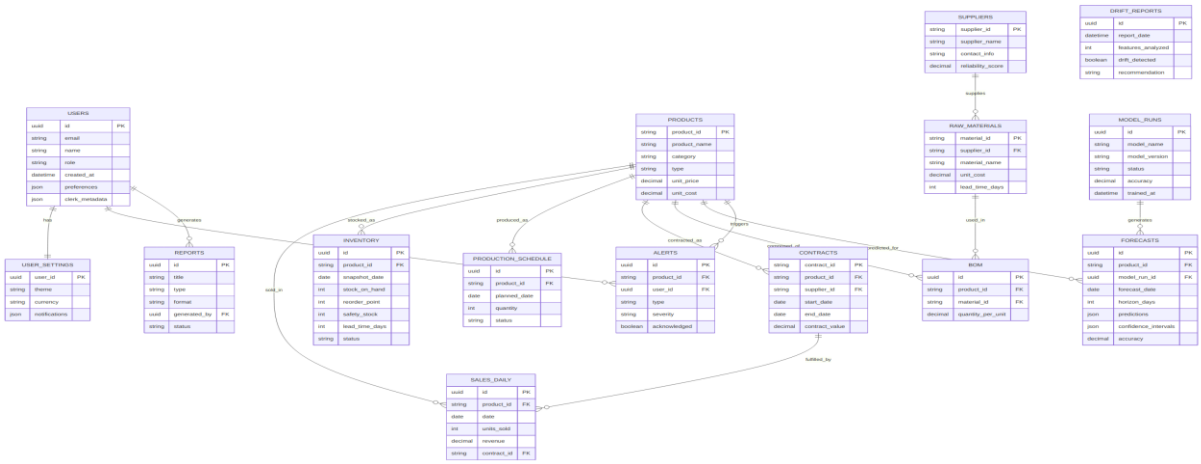


Figure 6.1 — Entity-relationship diagram covering the business domain and platform operational entities.

6.1 Source Datasets

Dataset	Description
products.csv	Product catalog (13 products across categories)
sales_daily.csv	Daily sales transactions, 2020–2024 (~15,000 rows)
inventory.csv	Daily inventory snapshots, 2020–2025 (~24,000 rows)
production_schedule.csv	Daily planned/actual production schedules
suppliers.csv	8 suppliers with reliability scores and lead times
contracts.csv	B2B contracts (25 records)
bom.csv	Bill of materials — product-to-raw-material mapping (40 rows)
raw_materials.csv	6 raw materials with cost and supplier links

An additional set of six enriched CSVs (demand, inventory, sales-monthly, product cost, and production) is derived from the raw sources for direct use by the forecasting and analytics services.

7. Core User Flow

The sequence below traces a representative session: sign-in, a forecast request, an inventory-optimization request, and an AI-insight request — the four calls that together drive most of the platform's day-to-day value.

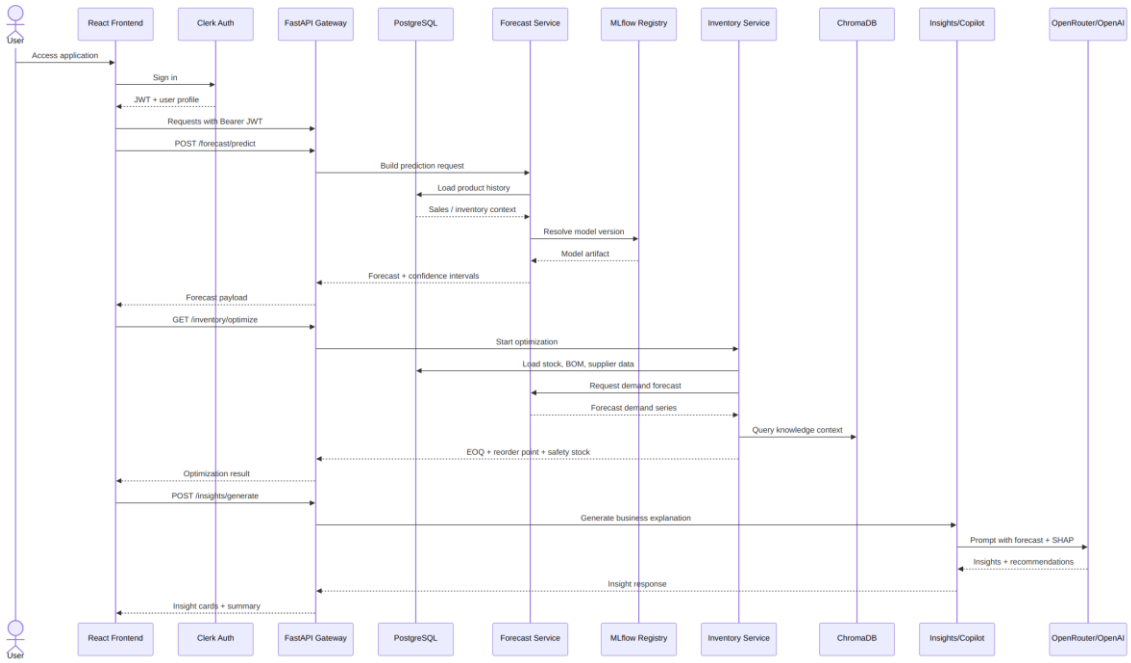


Figure 7.1 — Sequence diagram: authentication, forecasting, inventory optimization, and AI insight generation.

8. Technology Stack & API Surface

8.1 Technology Stack

Layer	Technology	Purpose
Frontend	React 18, TypeScript, Vite	SPA dashboard and UI
Styling	Tailwind CSS, shadcn/ui, Framer Motion	Neumorphism design system and animation
Auth	Clerk (@clerk/clerk-react)	JWT authentication, domain validation
State	React Query + Context API	Server state and global app preferences
i18n	react-i18next	English/Arabic with RTL support
Backend	FastAPI, Python 3.10+	REST API and business logic
Database	PostgreSQL (prod) / SQLite (dev)	Persistent data store
Cache	Redis	API response caching
ML model	XGBoost	Demand forecasting (R^2 = 0.9984)
Explainability	scikit-learn + SHAP	Feature selection and attribution
MLOps	MLflow + LangSmith	Model registry and agent tracing
Vector DB	ChromaDB	RAG knowledge base
LLM	OpenRouter (multi-model gateway)	Natural-language insights and copilot
Queue	Celery + Redis	Asynchronous task processing
Containers	Docker Compose (dev + prod)	Scalable deployment

8.2 Representative API Surface

The backend exposes 10+ routers and 30+ endpoints under /api/v1 (plus a top-level /auth surface). A representative sample:

Method	Endpoint	Access	Description
POST	/api/v1/forecast/predict	Manager+	ML prediction with confidence intervals
GET	/api/v1/inventory/optimize	Any authenticated	EOQ / reorder point / safety stock
POST	/api/v1/inventory/rag-query	Any authenticated	Inventory-specific RAG query
POST	/api/v1/insights/generate	Any authenticated	Statistical insight generation
POST	/api/v1/copilot/chat	Any authenticated	Orchestrated agent chat
POST	/api/v1/copilot/chat/stream	Any authenticated	Streaming (SSE) orchestrated chat

Method	Endpoint	Access	Description
GET	/api/v1/mlops/metrics	Admin	Model metrics and system resource stats
GET	/api/v1/mlops/langsmith	Admin	LangSmith agent tracing data
POST	/api/v1/system/reports/generate	Manager+	Per-type CSV report generation
GET	/auth/admin/users	Manager+	List platform users

8.3 Deployment Topology

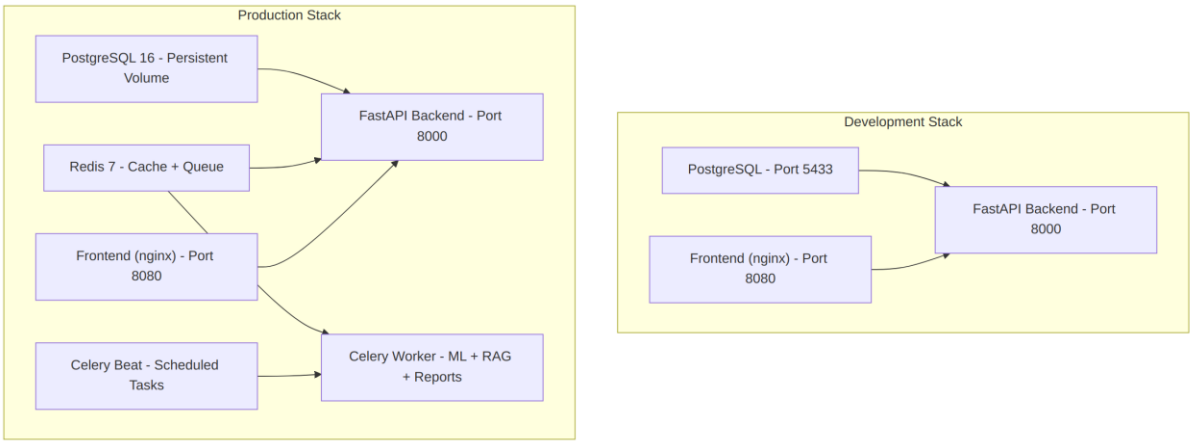


Figure 8.1 — Docker Compose deployment: development stack vs. extended production stack.

Service	Development	Production
PostgreSQL	Port 5433	Persistent volume
Redis	Not used	Cache + Celery broker
Backend	Port 8000	Port 8000
Frontend	Port 8080 (nginx)	Port 8080 (nginx)
Celery worker	Not used	ML / RAG / report tasks
Celery beat	Not used	Scheduled jobs

9. Business Case

Beyond the technical build, the team produced a formal business plan and a Power BI–ready market-sizing workbook (TAM/SAM/SOM, competitive landscape, and financial projections) to evaluate SupplyMind AI as a candidate SaaS product, not only as an academic deliverable.

9.1 Positioning

SupplyMind AI is positioned as a B2B SaaS decision-support platform for small and mid-sized manufacturing and industrial supply organizations — businesses that already generate operational data but lack advanced forecasting, explainability, and automation. The initial data model and feature set align most closely with industrial equipment manufacturing and adjacent supply-chain-heavy verticals.

9.2 Value Proposition

- Demand forecasting with multi-horizon predictions and confidence ranges.
- Inventory optimization via reorder point, safety stock, and order-quantity logic.
- Explainable AI insights that translate model output into business language.
- Risk and alerting workflows for stockouts, overstock, and demand anomalies.
- Operational and executive reporting.
- MLOps monitoring for drift, retraining, model quality, and system health.

9.3 Reference Benchmarks

Where the business plan cites external industry benchmarks rather than figures measured on this project's own data, the established sourcing standard for this team is McKinsey-published ranges:

Benchmark	Range	Status
Inventory reduction	20–30%	Industry benchmark (McKinsey) — planning reference, not measured on this platform
Stockout loss reduction	Up to 65%	Industry benchmark (McKinsey) — planning reference, not measured on this platform
Revenue lift	2–3%	Industry benchmark (McKinsey) — planning reference, not measured on this platform
Forecast accuracy	R^2 = 0.9984, WAPE = 1.51%	Measured — this project's evaluation set

> *Market-size figures vary significantly across vendors and data sources; any TAM/SAM/SOM figure quoted from the business plan should be read alongside its stated confidence level and source, not as an independently verified fact.*

10. Engineering Status, Known Gaps & Roadmap

10.1 Completed

- FastAPI backend with 10+ routers and 30+ endpoints.
- XGBoost demand forecasting model ($R^2 = 0.9984$).
- Clerk authentication with JWT verification and domain validation.
- RBAC with 4 roles and 19 permissions, with role names aligned consistently across frontend and backend.
- Guardrails middleware (input, output, rate limiting, tenant isolation).
- AI Orchestration Layer: intent-based routing to 5 specialized, logically isolated agents.
- RAG pipeline (ChromaDB, embeddings, hybrid vector + keyword search).
- Full LangSmith tracing across all orchestrator agents and service-level LLM call sites.
- Full English/Arabic i18n with RTL layout support (12 translation namespaces).
- Neumorphic design system with light/dark mode.
- Docker Compose for both development and production stacks.
- Celery worker for asynchronous ML, RAG, and report tasks.
- Per-type CSV report generation and deletion (5 report types).
- Command Center (Daily Mission Briefing): a 10-section operational overview wired to live API data, since extended to Reports, MLOps, and Alerts pages.
- Real-time supply-chain mapping nodes derived from backend data.
- Accessibility pass: keyboard navigation, ARIA labels, focus rings, screen-reader text.
- Viewer role hardened to strictly read-only (chatbot, write triggers, and generation privileges removed).

10.2 Pre-Commercial Gaps

The team's own technical review identifies the following as the primary gaps between the current build and a commercially deployable product:

Gap	Description
Billing	No subscription or usage-billing system yet exists.
Multi-tenant isolation	Tenant guardrails exist at the middleware level, but full data-isolation guarantees across customer organizations are not yet in place.
Dataset breadth	The platform currently runs against a single synthetic demo dataset; a commercial rollout needs validation against real customer data.
Licensing / IP	The repository currently carries an academic-only license, which requires an IP reassignment agreement from all contributors before any commercial use.
Security review	Outstanding security-review items remain open ahead of a production launch.

Gap	Description
Documentation debt	The repository's ARCHITECTURE.md file is stale relative to the current codebase layout and should be retired or rewritten in favor of README.md and architecture-diagrams.md.

10.3 Future Improvements

- Multi-warehouse optimization engine.
- Real-time streaming forecasts (Kafka integration).
- ERP system integrations (SAP, Oracle).
- Multi-region, multi-currency support.
- Scenario simulation and what-if analysis.
- Advanced anomaly detection using isolation forests.
- Mobile app (React Native).
- CI/CD pipeline via GitHub Actions.
- Fully automated model-retraining triggers.

11. Team & Component Ownership

The six-person team divided the system along clear functional lines, with the backend/deployment lead acting as the integration point for every other workstream.

Owner	Area	Primary Files / Components
Rahma (Data Analyst)	Data & Dashboard	data/, data/enriched data/, Dashboard.tsx, dashboard components
Karim (ML Engineer)	Model Training	ml_platform/models/, demand_model_pipeline.pkl
Kenzi (LLM Engineer)	AI Insights	AIInsights.tsx, backend/llm/
Ali Ehab (RAG Engineer)	Inventory & RAG	Inventory.tsx, inventory components, RAG system
Ali S. (MLOps Engineer)	MLOps & Knowledge	MLOps.tsx, backend/knowledge/
Ibrahim (Team Lead)	Backend & Infra	backend/ core, docker-compose*.yml, backend/auth/

The Forecasting page is jointly owned between the ML Engineer (model output) and the Team Lead (API integration), reflecting the natural handoff point between the model layer and the serving layer.

11.1 Contribution Workflow

The team follows a feature-branch workflow: branches are cut per contributor per feature (feature/<name>/<feature-name>), and every pull request targeting main requires one review from the team lead before merging.

12. Getting Started

12.1 Prerequisites

- Node.js ≥ 18.0.0, npm ≥ 9.0.0
- Python ≥ 3.10
- Docker ≥ 24.0.0 (optional, for the containerized setup)

12.2 Quick Start (Docker)

```
git clone https://github.com/IbrahimAbdelsattar/SupplyMindAI.git
```

```
cd SupplyMindAI  &&  cp .env.example .env  &&  docker compose up -d
```

Frontend: http://localhost:8080 · Backend API: http://localhost:8000 · API docs: http://localhost:8000/docs

12.3 Manual Setup

```
Frontend: npm install && npm run dev  (serves on :8080)
```

```
Backend: python -m venv .venv && source .venv/bin/activate && pip install -r backend/requirements.txt && uvicorn backend.main:app --reload --port 8000
```

12.4 Key Environment Variables

Variable	Purpose
VITE_CLERK_PUBLISHABLE_KEY / CLERK_SECRET_KEY	Clerk authentication
CHATBOT_API_KEY / LLM_REASONING_API_KEY / RAG_API_KEY	OpenRouter LLM access per feature
DATABASE_URL	PostgreSQL connection string
REDIS_URL	Redis cache/queue connection
LANGCHAIN_TRACING_V2 / LANGCHAIN_API_KEY	Optional LangSmith observability
MODEL_PATH / DRIFT_THRESHOLD	MLOps: model artifact path and drift sensitivity

13. Summary

SupplyMind AI demonstrates a complete, production-shaped approach to an AI-driven supply chain problem: a trained and evaluated forecasting model, an explainability layer that makes that model's output defensible, a guardrailed LLM integration that makes it safe to expose to end users, and a full-stack application that makes it usable by the people who actually run supply chain operations. The remaining path to a commercial product is well scoped — billing, multi-tenant isolation, broader data validation, and licensing — and is documented as such rather than glossed over.

This documentation is intended to travel with the project: as a graduation submission today, and as the onboarding reference for the next phase of the team's work.

— SupplyMind AI Team